

OrbitOS

系统全景 · 设计取舍 · 质疑应答图谱

一个把 CRM 与 ERP 打通、由 Agent 主动发现跨部门风险并协同处理的企业运营系统。
本册用 16 张流程图回答两件事：**这套系统是怎么运转的**，以及 **每一个会被追问的地方，我的答案和证据在哪里**。

主 张 01

规划先于执行，而且可见

Planner 先产出结构化 JSON 计划，界面立刻渲染成带勾选框的清单，然后才开始调工具。让规划可见本身就是功能。

主 张 02

权限是物理约束

未授权的工具不会进入发给模型的 tools 数组，执行前还要再查一次角色。
一个有 3% 概率失效的权限系统等于没有。

主 张 03

写操作停下来等人

4 个写工具执行前一律暂停，确认卡上的金额与到货日由代码从数据库现算。确认卡不能由被审查的对象生成。

全 册 统 一 语 义 · 谁 做 的 决 定

- LLM 决定

概率性 · 可能出错
- 代码决定

确定性 · 可测试
- 人决定

HITL 确认点
- 被拦下

权限 / 幻觉 / 写操作
- 数据流

读 / 写 / 事件
- 已知缺口

主动交代 · 未实现

本 册 目 录

01 一页看懂：问题 → 赌注 → 主张 → 证据	Executive	02 问题地图：那堵墙立在哪里	Why	03 系统总架构：谁跟谁说话	Architecture
04 剧本 A 全链路时序：17 步不省略	End-to-end	05 Planner-Executor：为什么不用纯 ReAct	Trade-off	06 工具边界：一个反例值 40 行代码	Trade-off
07 权限三道闸：物理约束不是君子协定	Security	08 幻觉六道防线：各拦一类，互不替代	Trust	09 HITL：把注意力花在不可逆那一步	Product
10 失败设计：宁可显眼地失败	Resilience	11 第二条链路：飞书群 ↔ OrbitOS	Channel	12 质疑应答图谱 ①：数据 / 幻觉 / 权限 / 确认 / 模型	Q&A
13 质疑应答图谱 ②：工具 / 评测 / 落地 / 口径 / 审计	Q&A	14 评测体系：为什么先盯工具选择准确率	Metrics	15 V1 → V2 演进：触发条件而不是路线图	Roadmap
16 决策总表：选了什么 · 代价 · 何时改选	Summary				



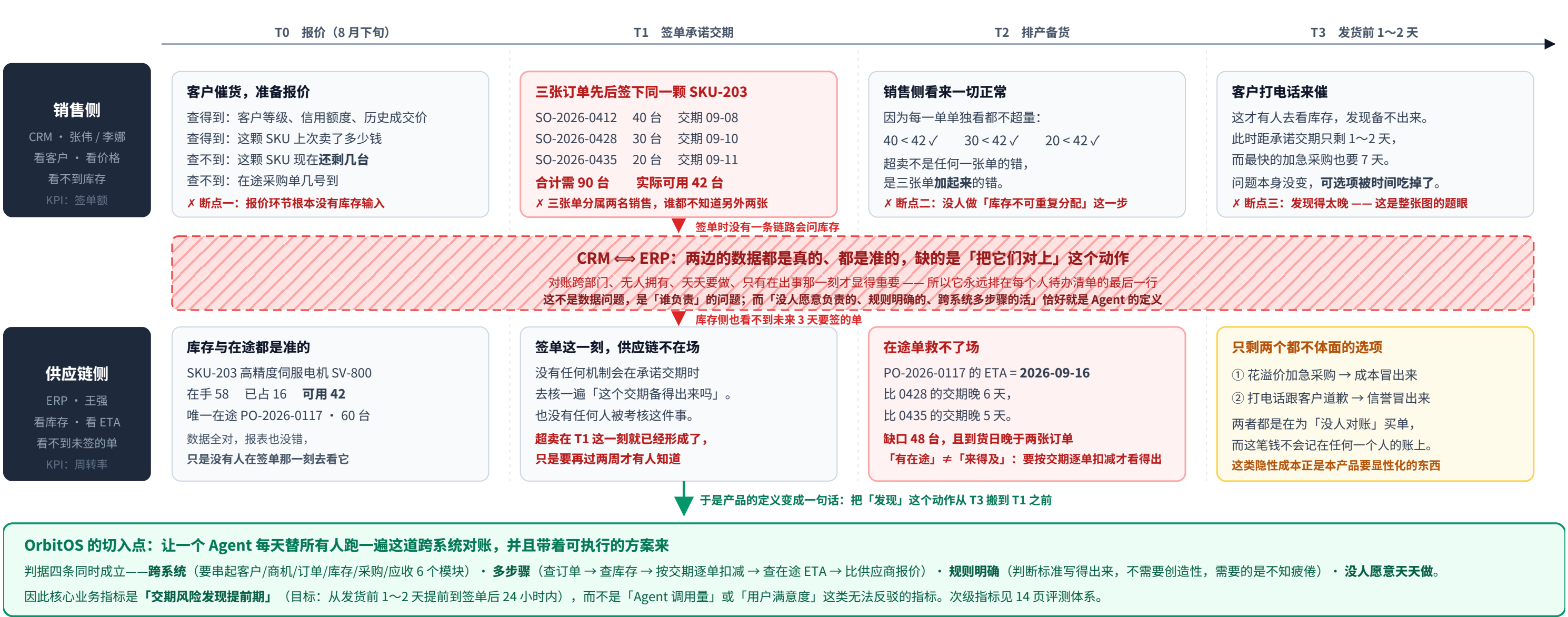
四段推演读法：左边两列是「为什么做」，右边两列是「怎么做到、怎么被验证」。每一条主张在右侧都有一个可以当场点开的机制对应，没有只停留在说法上的主张。

6 业务模块 客户/商机/订单/库存/采购/应收	15 = 11 读 + 4 写 Agent 工具总数 CRM 4 / ERP 6 / 分析 3 / 写入 2	9 · 11 · 10 · 15 四角色实际可用工具数 销售代表/销售总监/供应链/CEO	100% 写操作拦截率 4 个写工具执行前一律暂停	340 单元测试用例（20 文件全绿） 另有 1 个需线上 endpoint 的 e2e 脚本	9 我主动交代的实现缺口 网页链路 6 条 + 飞书链路 3 条（见 11 / 13）
--	--	---	---	---	---

PROBLEM MAP

02 那堵墙立在哪里：一次超卖是怎么长出来的

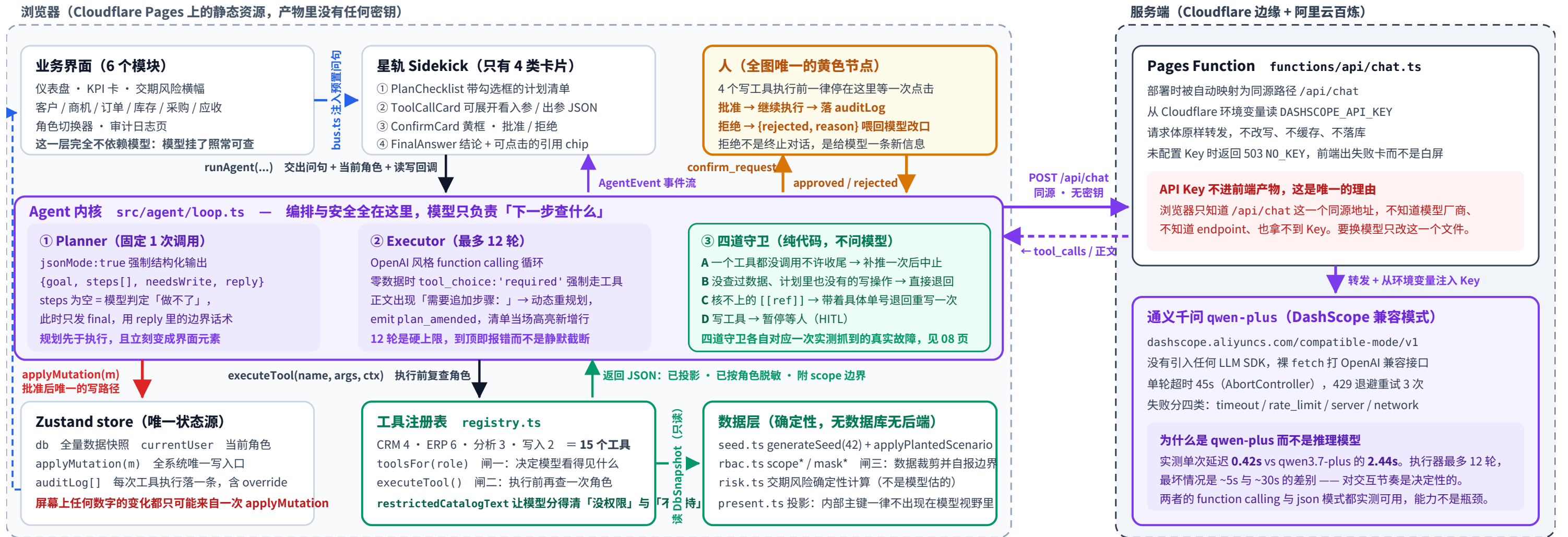
B2B 分销商最贵的一堵墙立在销售和供应链之间。下图按时间顺序还原一次超卖的完整生长过程——注意红色断点全部发生在「签单」之前，而人发现它的时刻在「发货前」，中间隔着的正是本产品要抢回来的提前期。



03 系统总架构：谁跟谁说话，说的是什么

每根箭头都标了具体的函数名或事件名，不是「相关」。

紫 = 模型决定 · 绿 = 代码决定 · 黄 = 人决定 · 红 = 写入 · 蓝虚线 = 数据流



我主动交代的第一个缺口：网页这条链路的三道权限闸，目前全都跑在浏览器里

这是一个无后端的演示系统——数据由 generateSeed(42) 在浏览器里确定性生成，工具注册表与 executeTool 也在浏览器里。懂技术的人打开 devtools 改掉 currentUser.role，三道闸一起失效。

生产环境的正确做法：把工具注册表、executeTool 与整个数据层搬到服务端，角色从会话取而不是从前端状态里取，前端只保留 UI 与事件渲染。分层结构一行都不用改。

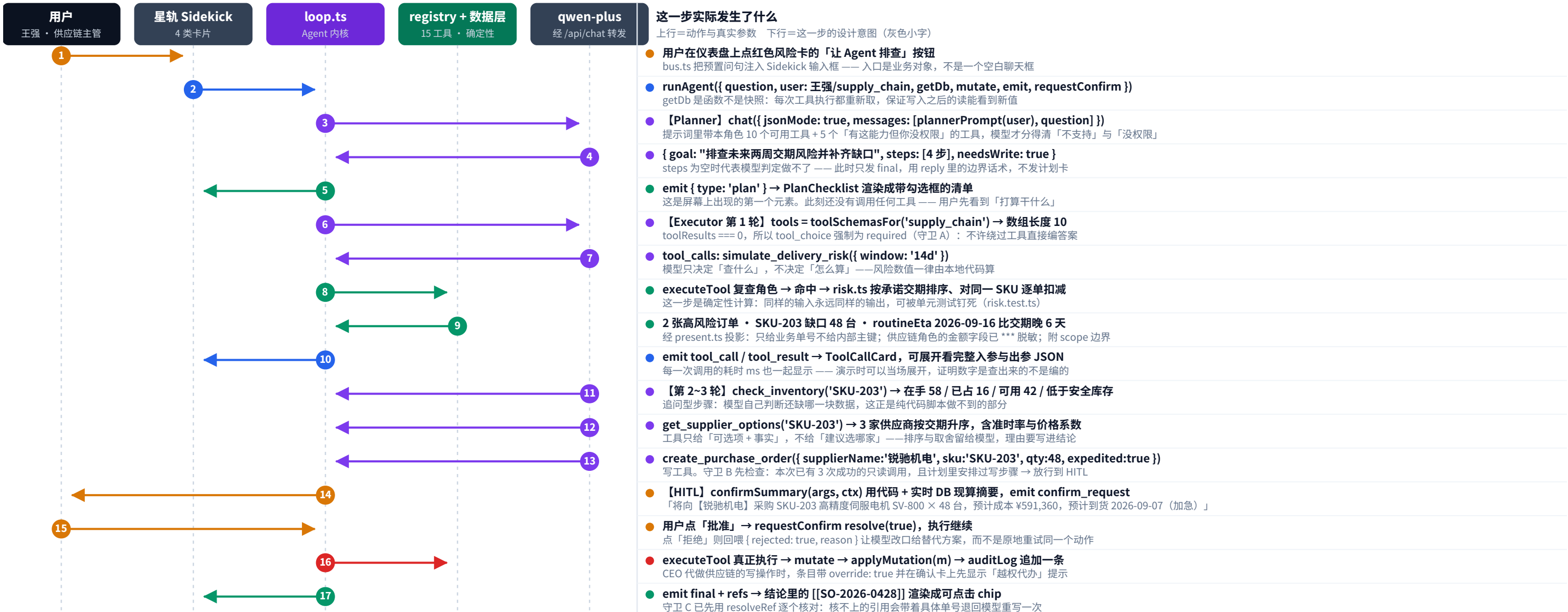
这一点已经被第二条链路验证过了：飞书那条链路里，同一份 loop.ts 一行没改就在 Worker 里跑通，身份改由服务端 resolveUser 查出来、不由调用方传（见 11 页）。

把这句话直接写在架构图上、而不是等人问出来，是因为：一个说不清自己边界在哪的系统，比一个边界画得保守的系统更不可信。网页链路另外五个缺口见第 13 页，飞书链路的三个见第 11 页。

读法：纵向是一次问询的下沉方向（展示层 → 内核 → 能力与数据），横向是同一层里的职责切分。整张图只有一个黄色节点（人），只有一条红色写路径（applyMutation），并且这条红线的上游必须先经过黄色节点——这就是「写操作强制人工确认」在架构上的样子，而不只是文档里的一句承诺。

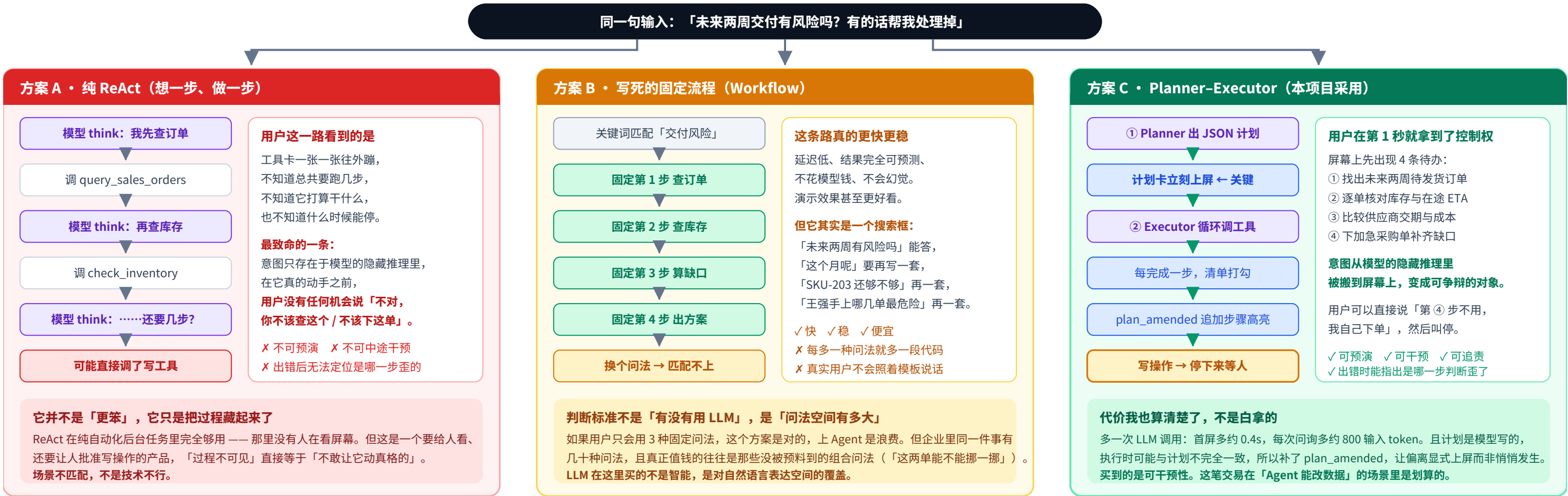
04 剧本 A 全链路时序：17 步，一步不省

这是现场演示的那 90 秒在系统内部的完整展开。
 每一步都标了真实的函数名与真实的参数值，圆点颜色表示这一步是谁做的决定。



05 为什么是 Planner-Executor，而不是纯 ReAct 或写死的流程

同一个问题「未来两周交付有风险吗，帮我处理」，在三种架构下用户在第 1 秒看到的东西完全不同。
这一页比较的不是技术优雅度，是**可干预性**：用户能不能在代价发生之前叫停。



TRADE-OFF • 2

06 工具边界：哪些事绝不交给模型，以及为什么

这是「SKU-203 缺口 48 台」这个数字的全部来历——一张按承诺交期排序、逐单扣减的台账
起始可用量 42 台（在手 58 – 已预留 16）。规则：交期早的先占用库存，占完为止；后面的单拿到多少算多少。

订单号	承诺交期	需求	扣减前可用	本单缺口	扣减后可用
S0-2026-0412	2026-09-08	40 台	42 台	0	2 台
S0-2026-0428	2026-09-10	30 台	2 台	28 台	0
S0-2026-0435	2026-09-11	20 台	0	20 台	0
合计		90 台	42 台	48 台	唯一在途 09-16 才到

注意第 2 行：「扣减前可用」是 2 而不是 42——库存不能被两张订单重复分配。
这一个字的差别，正是「90 台需求 > 42 台库存」这件事在单看任何一张订单时都看不出来的原因。



07 权限：三道闸，以及三条被堵死的绕行路

15 × 4 权限矩阵 —— 直接从 src/agent/tools/*.ts 的 allowedRoles 读出来的，不是文档抄的

工具	模块	销售代表	销售总监	供应链	CEO
query_customers	CRM	●	●	●	●
get_customer_detail	CRM	●	●	●	●
query_opportunities	CRM	●	●	○	●
create_followup_task	CRM	写 ●	●	○	●
query_sales_orders	ERP	●	●	●	●
get_order_detail	ERP	●	●	●	●
check_inventory	ERP	●	●	●	●
query_purchase_orders	ERP	○	○	●	●
get_supplier_options	ERP	○	○	●	●
create_purchase_order	ERP	写 ○	○	●	●
query_receivables	分析	○	●	○	●
aggregate_metrics	分析	○	●	○	●
simulate_delivery_risk	分析	●	●	●	●
update_order_promise_date	写入	写 ●	●	○	●
reserve_inventory	写入	写 ○	○	●	●
可见工具数		9	11	10	15

黄条 = 写入工具（4 个）。●=可用 ○=不可用。这张表没有一个「管理员总开关」：CEO 的 15 不是特权位，而是 15 行里每一行都单独把 'ceo' 写进了 allowedRoles —— 加一个工具就必须重新回答一次「谁能用它」。

同一个角色变量，在一次问询里被查了三次



补充一：CEO 的 15 个工具不是「无限权」，是「越权可做但必须留痕」

CEO 调 `create_purchase_order` 这类本该由别的岗位执行的写操作时，`overrideNoticeFor()` 会在确认卡最上面加一行：

「越权代办：该操作通常由『供应链经理』执行。你以 CEO 身份直接执行，操作会完整留痕。」

同时 `auditOf(..., override=true)` 把这一次标成越权条目写进审计日志。**设计立场：老板要越权是组织现实，产品不该假装它不存在；**该做的是让越权变成一件**有成本、有记录、事后能被问责**的事，而不是把它伪装成一次普通操作。

这条提示只在 `role === 'ceo'` 且该写工具的 `WRITE_TOOL_OWNER` 不是 `ceo` 时才出现 —— 它是一条**判断，不是一句万能提示语**。

CEO 自己那份工作不会被打上越权标记，否则这行提示很快会被当成噪音，一路点过去。

补充二：拒绝的措辞本身也是产品问题 —— `restrictedCatalogText(role)`

闸一把工具藏起来之后出现了一个副作用：销售代表问应收，模型看不到那个工具，于是回了一句

「当前系统不支持查询应收账款。」—— 这是一句事实错误的话：系统支持，是你无权。

修法是把「本系统有哪些能力、其中哪些你这个角色用不了」单独喂给规划器。「能力外」和「权限外」必须说得不一樣：前者让用户放弃，后者让用户知道该去找谁申请。一个说错了的拒绝，比拒绝本身伤害更大。

这是我把权限当成**产品问题**而不只是安全问题来做的地方：安全关心的是有没有被拦住，产品还得关心被拦住的人接下来怎么办。

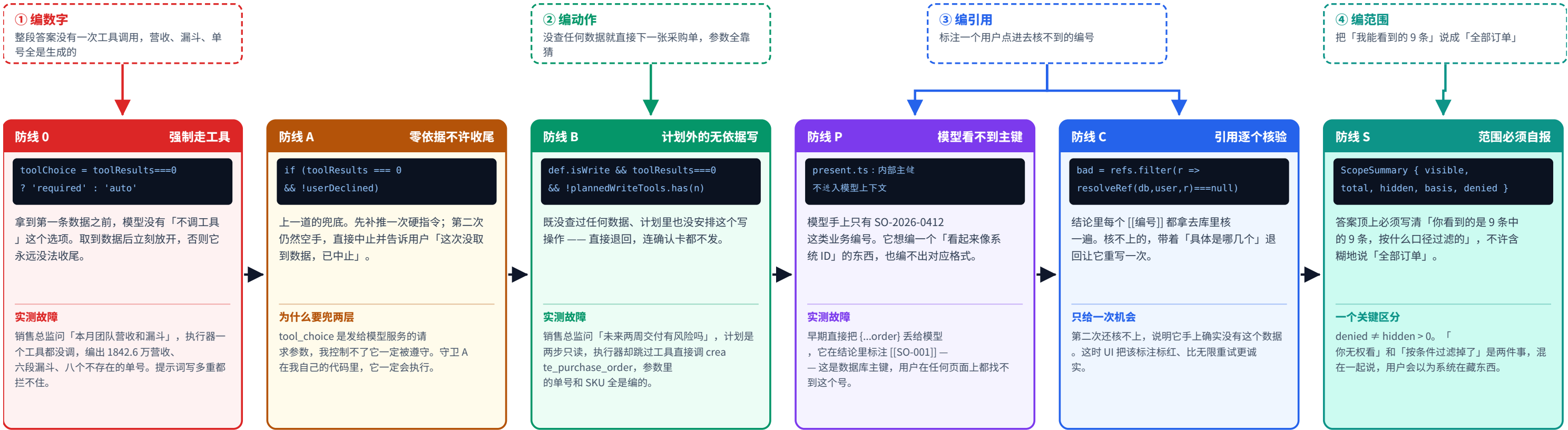
所以拒绝文案里一定带上「这件事归谁」—— 第 09 页确认卡上的越权提示，用的是同一份 `WRITE_TOOL_OWNER` 归属表。

我承认的边界：网页那条链路的三道闸目前跑在浏览器里（见第 03 页的信任边界）；飞书那条链路的身份判定已经在服务端（见第 11 页）。这是一个**面试演示项目**的合理取舍——它演示的是权限该被检查几次、在哪几层检查、拒绝时该说什么；接真实后端时，闸二与闸三必须原样搬到服务端，浏览器里的那份降级为「少发一次注定被拒的请求」的体验优化。

08 幻觉：六道防线，每一道都是被一次真实故障逼出来的

这一页不是「我们很重视幻觉」的表态，是六段代码和六次实测复现。
共同点：全部写在模型之外，没有一道依赖模型「守规矩」。

一次问询里，模型实际会编的四种东西 —— 它们不是「偶尔出错」，是四种**形态不同、要用不同办法堵**的失败



③ 编引用

标注一个用户点进去核不到的编号

防线 B 计划外的无依据写

```
def.isWrite && toolResults===0
&& !plannedWriteTools.has(n)
```

既没查过任何数据、计划里也没安排这个写操作 —— 直接退回，连确认卡都不发。

实测故障

销售总监问「未来两周交付有风险吗」，计划是两步只读，执行器却跳过工具直接调 create_purchase_order，参数里的单号和 SKU 全是编的。

④ 编范围

把「我能看到的 9 条」说成「全部订单」

防线 P 模型看不到主键

```
present.ts : 内部主键
不进入模型上下文
```

模型手上只有 SO-2026-0412 这类业务编号。它想编一个「看起来像系统 ID」的东西，也编不出对应格式。

实测故障

早期直接把 {...order} 丢给模型，它在结论里标注 [[SO-001]] —— 这是数据库主键，用户在任何页面上都找不到这个号。

防线 C 引用逐个核验

```
bad = refs.filter(r =>
resolveRef(db,user,r)===null)
```

结论里每个 [[编号]] 都拿去库里核一遍。核不上的，带着「具体是哪几个」退回让它重写一次。

只给一次机会

第二次还核不上，说明它手上确实没有这个数据。这时 UI 把该标注标红、比无限重试更诚实。

防线 S 范围必须自报

```
ScopeSummary { visible,
total, hidden, basis, denied }
```

答案顶上必须写清「你看到的是 9 条中的 9 条，按什么口径过滤的」，不许含糊地说「全部订单」。

一个关键区分

denied ≠ hidden > 0。「你无权看」和「按条件过滤掉了」是两件事，混在一起说，用户会以为系统在藏东西。

六道的排列不是随机的：越靠前，代价越低

防线 0 是一个请求参数，防线 A 是一个 if，防线 B 是一次 return，防线 C 要多跑一轮模型（多一次延迟、多一次 token）。

能用一个参数解决的，绝不用一次重试解决；
能用一次重试解决的，绝不留给用户去发现。

最后一道兜底在界面上：重写后仍核不上的标注，UI 标红并禁用跳转。
我选择显式承认「这一条我核不上」，而不是悄悄删掉它 —— 删掉之后用户看到的是一段完全通顺的答案，没有任何线索知道这里出过问题。

第七类幻觉：它不编数据，它编「系统的行为」—— 这一道我至今只能用提示词顶住

复现：Agent 老老实实调了工具、每个单号都核得上，却在中间写了一句 ——

「2025 年订单已归档至总部财务系统。」 系统里根本没有「归档」这回事。

它是看到工具返回里的 hidden 计数之后，给这个数字现编了一个听起来很合理的机制。前六道全拦不住 —— 因为「原因」不是一个能用代码核对的对象。
危害不比编单号小：**用户会照着这句话去行动** —— 去财务系统找、去找 IT 要权限、在会上说「我们的历史数据在财务那边」。
而且只写「不许编造原因」拦不住：模型不认为自己在编造，它认为自己在合理推断。得把它最可能说出口的那三句原话点名摆出来，它才对得上号。

真正的修法方向（这一版没做，记在文档里）：把 hidden 这类字段的含义也塞进工具返回 ——
{ "hidden": 70, "hiddenReason": "不在你的权限范围内，无其他原因" }。给它一句可复述的话，正是第四类防线被证明有效的那条思路。

读这一页的正确方式：**从下往上看代价**。六道防线里没有一道是「加一段提示词让模型注意」—— 提示词是建议，代码是约束；一个会在演示当天失灵的机制，不算机制。

09 人工确认：这张卡上的每一个字，都不是模型写的

「为什么还要人点一下？」——因为写操作的代价不可逆，而模型的置信度不可信。
但真正的设计难点不在「弹不弹卡」，在卡上写什么、由谁写。

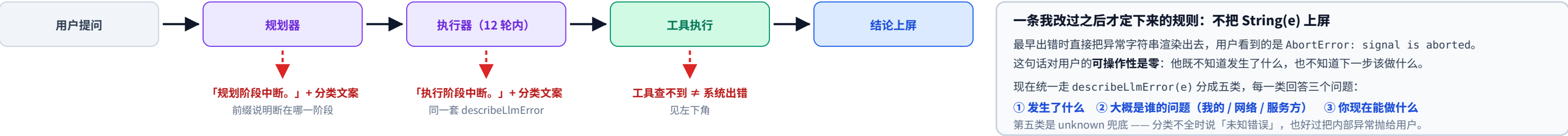
① 确认卡解剖 —— 以 CEO 身份下加急采购单为例



10 失败设计：一个 Agent 产品的下限，写在它出错的那一屏上

演示顺利时看不出产品经理的水平，**出错时才看得出**。
所以这一页画的是：它会在哪几个地方断，断了给用户看什么，以及给不给出路。

一次问询的完整链路上，有六个地方会断——每一个都有一句专门写给它的文案，没有一句是「出错了，请重试」



timeout模型 45 秒没回

TIMEOUT_MS = 45000

用户实际看到的话

「模型 45 秒没有响应，可能是网络波动或模型服务繁忙。通常重试一次就能跑通。」

给的出路

给「重试」按钮，并提示：左侧业务页面的数据不经过模型，现在就能查。

rate_limit被限流

已自动退避重试 3 次

用户实际看到的话

「模型服务当前限流，已自动退避重试 3 次仍未成功，请稍后再试。」

给的出路

先自动退避 3 次，失败才上屏。这也是换掉上一个模型的原因：免费额度两人同时演示就挂。

server模型服务报错

HTTP 4xx / 5xx

用户实际看到的话

「模型服务返回 HTTP 401。通常是 API Key 未配置或已失效。」

给的出路

401/403 单独说。这是运维问题，让用户点重试是浪费他的时间。

network连不上

fetch 抛错 / 404

用户实际看到的话

「连不上模型服务，可能是网络不通，或 /api/chat 未部署。」

给的出路

把「你的网」和「我的部署」两种可能都写出来，用户才知道该刷新还是该找人。

NO_KEY服务端没配密钥

Function 返回 503

用户实际看到的话

归入 server 类。密钥只存在 Cloudflare 环境变量里，前端代码与仓库里都没有。

给的出路

宁可线上偶尔 503，也不把 Key 打进前端包——打进去等于公开。

MAX_TURNS转了 12 轮还没完

for turn < MAX_TURNS = 12

用户实际看到的话

「已达最大执行轮数 12，任务未完成。」

给的出路

必须有硬上限：没有它，一个绕不出来的循环会一直烧 token，而用户只看到转圈。

工具「查不到」是一个正常结果，不是一次报错

工具查无此单时返回 { found:false, reason:'未找到订单「S0-xxxx」，或当前角色无权修改' }，库存不够时返回 { found:false, reason:'SKU-203 可用库存不足：现有可用 42 台，需要 48 台' }。

两个关键点：

① 带 reason 而不是只回一个 false——模型拿到理由才能改口径重查，或者如实告诉用户为什么没有；

② 「查不到」不走错误通道——它不该弹红条，因为系统运转完全正常，只是这个业务事实不存在。混在一起，用户会以为系统坏了。

我删掉了唯一一个能保证演示不翻车的功能：录播模式

曾经有一个开关，能把一整轮问询的事件流记录下来原样回放。有了它，面试当天断网也能演。我把它彻底删掉了，不是注释掉，是从代码库里删干净。

理由：只要这个开关还在，任何一次演示都可以被质疑「你放的是录像还是真的在跑」。而这个项目要证明的恰恰是「它真的会跑，也真的会失败」——一个不会失败的演示，什么都没证明。留一个「保底能演」的开关，就等于承认演示是给人看的表演。

代价我认：现场如果模型限流，我就当场展示第 3 张卡那句限流文案。这也是内容的一部分。

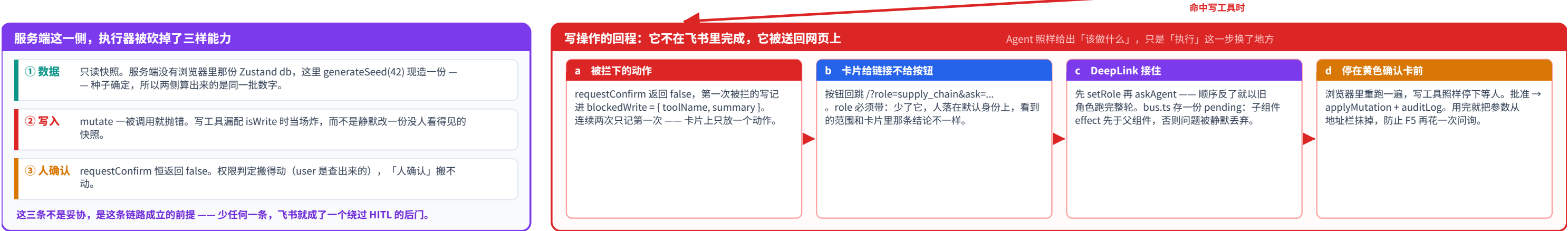
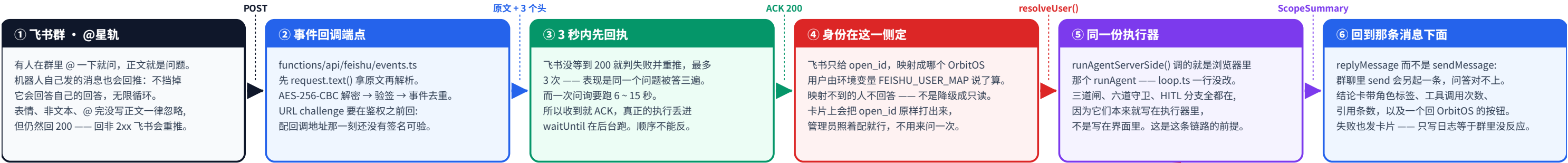
把这六张卡连起来看是一条产品原则：**错误信息的收件人是用户，不是开发者**。所以它必须回答「谁的问题、我现在能做什么」，而不是把一段异常原样转发出去——转发异常，是把排查成本从我这里推给了用户。

11 第二条链路：飞书群 ↔ OrbitOS，同一份执行器

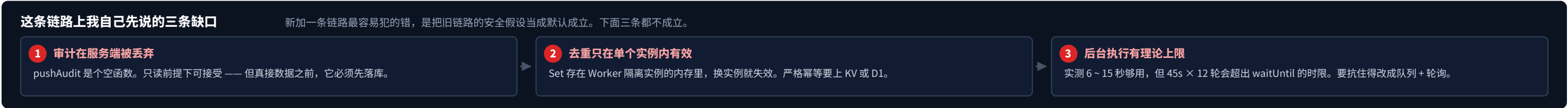
同一份 loop.ts 一行没改，在 Cloudflare Worker 里又跑了一遍。
换掉的只有三样：数据从哪来、写操作能不能批、结论发到哪去。

入站：群里的一句话 → 一条带权限边界的结论

这条链路上没有任何一步的判断是飞书那边给的：身份是查出来的，权限是执行器算的，写操作是被拒的。



协议层的四个坑：每一个都不会报有用的错，只会「就是不通」



上半页横向读是入站：群里的一句话 → （验签）→ （先回执）→ （在服务端定身份）→ （同一份执行器）→ 一条带范围声明的结论。下半页左边是这条链路上执行器被拿掉的三样能力，右边是写操作的回程 —— 它不在飞书里完成，红色的链条最后一定落回网页上那张黄色确认卡的前面。这是第 07 页那三道闸第一次真正跑在服务端：身份是 resolveUser 查出来的，不是调用方传进来的。

Q & A · 1 / 2

12 质疑应答图谱 ① 追问 1 – 5：真实性 · 幻觉 · 权限 · HITL · 成本

每一行都按同一个顺序回答：质疑 → 机制 → 现场能验证的证据 → 我自己先说出来的边界。
第四列是这张表里最重要的一列 —— 没有边界的回答，听起来像销售话术。

面试官的质疑	我的回答（机制在第几页）	现场就能验证的证据	我主动承认的边界
1 数据是真的吗？ 「你这些客户名、单号，是不是随便编的？」	全部虚拟生成，但生成方式是确定性的。generateSeed(42) 用 mulberry32 产出 48 客户 / 90 商机 / 60 SKU / 160 销售订单 / 55 采购单 / 120 应收，同一个种子永远产出同一份数据。演示要的冲突链不靠随机碰运气：随机之后由 applyPlantedScenario() 硬编码覆盖出「SKU-203 可用 42 台 / 三张订单共需 90 台 / 唯一在途采购单晚到 / 两家供应商一个快一个便宜」这条链。	seed.test.ts 里有一条「相同种子产出完全一致」的断言，另有 10 条断言把埋雷链逐项钉死 —— 改坏了，测试当场变红。	真实数据在演示阶段是负债不是资产：敏感、拿不到、还不可复现。这个取舍我认。
2 幻觉怎么防？ 「大模型会瞎编，你怎么保证它不编？」	见第 08 页六道防线。一句话：提示词是建议，不是约束。我把「先查后答，没有例外」提到规则第 0 条、写了三行强调，照样复现出零工具调用的整段编造。真正的修法是把「该不该」改成「能不能」—— 只要还没拿到任何工具结果，请求就带 tool_choice:'required'，模型这一轮在协议层面没有输出正文的选项。之上再叠五道：越靠前越接近「让它没法这么做」，越靠后越接近「做了也发不出去」。	打真实模型的端到端回归跑 14 条场景，逐条核对结论里每个 [[编号]] 能不能在库里查到：15/15 通过，0 条引用核不上。	恒定的是「核不上 0 条」这个判据，不是引用条数 —— 模型非确定，同一问题两次跑出的工具链都不同。第七类幻觉（编造系统行为）仍只能靠提示词顶。
3 权限能被绕过吗？ 「让它假装自己是 CEO，是不是就查到了？」	工具层双检 + 数据层作用域，见第 07 页。这里有个我真踩过的坑值得讲：simulate_delivery_risk 对四个角色都开放，但它收一个 orderNos 参数 —— 直接按订单号查库的话，销售代表就能读到别人的订单。工具级权限过了，数据级权限被绕过了。修法是传入的订单号先过一遍 scopeOrders，越权的过滤掉，并把 deniedCount 显式回报给模型：静默丢弃更糟，模型会以为自己拿到了完整答案。	registry.test.ts 直接钉住第二层：executeTool('aggregate_metrics', ..., 张伟) 断言 ok=false、error='PERMISSION_DENIED'。另两条覆盖全越权与混合传入。	三层目前全在浏览器端，防的是模型不是用户。生产环境必须把注册表和 executeTool 搬到服务端 —— 架构不用变，要变的是信任边界的位置。
4 为什么非要人点一下？ 「加个确认弹窗，不就是个交互细节吗？」	两个理由，第二个更重要。一是写操作不可逆：查错了刷新就行，写错了要走退单流程 —— 所以只拦写不拦读。读也拦的话一次会话要点五六次确认，人会退化成点击机器，然后第六次那张 ¥591,360 的采购单也被无脑点过去。拦得越多，真正该拦的那一次越拦不住。二是拒绝率是产品质量信号：拒绝率高不说明安全机制有效，说明 Agent 在提用户不想要的方案。	理想状态是拒绝率低、但确认卡停留时长不低 —— 人认真看了才同意。这两个指标写进了 PRD 的核心指标，不是事后补的。	已知缺口：被拒绝的调用当前没进 auditLog，而拒绝率恰恰要靠它算。指标定义和埋点实现脱节了，这是上线前要补的一行代码。
5 用了什么模型、多少钱？ 「成本你算过吗？还是只做了个能跑的 demo？」	qwen-plus（DashScope OpenAI 兼容模式），temperature 0.2，原生支持 function calling 与 tool_choice:'required'，国内直连；经 Cloudflare Pages Function 代理，API Key 只存在于环境变量里，不进前端产物。第一版是 GLM-4.5-Flash，换掉的原因不是能力而是免费档速率上限：一次问询 = 规划 1 次 + 执行最多 12 次，两个人同时演示必然撞限流。选 qwen-plus 而不是推理型号，是实测单次延迟 0.4 2s 对 2.44s 的差异。	打线上接口实测：剧本 A（8 个工具、含一次 HITL 写）墙钟 29.5s，输入 18,074 / 输出 1,181 token；最便宜的单跳查询 6.8s / 7,747 输入；最贵一条 41,478 输入。	输入 token 是输出的 10–30 倍（工具返回的 JSON 每一轮都在重发），优化成本的着力点在这里，不在压缩提示词。Planner / Executor 的分段延迟与跑间方差仍未拆测，我不填估算值。

这五行的共同结构不是偶然：先承认问题成立，再给机制，再给可当场跑的证据，最后主动交出这套机制管不到的地方。一个说不出自己边界在哪的方案，通常意味着还没被真正用过。

面试官的质疑	我的回答（机制在第几页）	现场就能验证的证据	我主动承认的边界
6 工具为什么切成 15 个？ 「为什么不做一个万能的 query(entity, filter)? 」	→ 15 个 = 只读 11 + 写入 4（CRM 4 / ERP 6 / 分析 3 / 写入 2）。三条切分原则：① 一个工具对应一个业务问题 —— 万能 query 会同时毁掉四样东西：allowedRoles 没地方挂、脱敏没地方做、调用准确率掉、审计日志失去语义；② 确定性计算不交给 LLM，40 行纯函数永远算出 48；③ 粒度的上限由人决定 —— check_inventory 收 SKU 数组，否则 7 张订单最坏 21 次调用，撞穿 12 轮上限；而复合工具会让 H ITL 没地方插。	→ 15 个 ToolDef 在 registry.ts 里一眼可数；切角色，可用工具数当场从 15 变成 9 / 11 / 10。粒度那条现场能算：7 张订单 × 3 个 SK U = 21 次 > MAX_TURNS 12。	→ 「工具的粒度上限就是人能介入的最小粒度」这条，是我从 HITL 反推出来的，不是先验原则。工具真到几十个之后要不要分层路由，我没验证过。
7 怎么证明它是好的？ 「demo 跑通了，不等于系统是好的。」	→ 离线 30 条标注测试集，分层：单工具直查 8 / 多工具串联 8 / 权限拒绝 5 / 空结果 4 / 写操作 5。标准答案由代码算出来（直接调 executeTool），不是我手写的。离线三指标：工具选择准确率 / 参数正确率（只在选对的前提下算） / 结论事实一致率。在线四指标：人工确认拒绝率、追问率、任务完成率（分母剔除权限拒绝）、单次会话平均工具调用数（双边指标：太高是绕圈，太低是没查够）。	→ 为什么工具选择准确率排第一：它在因果链上游、客观可自动化、能归因到具体的工具描述、可以当稳定回归护栏。当前的手动关卡是端到端 15/15 。	→ 盲区很明确：工具全选对、参数也全对，但把返回的 48 读成 40 —— 这类只有结论事实一致率能抓，而它恰恰是三个指标里最难自动化的一个。
8 接真实 ERP 要改多少？ 「这全是假数据，真接上去是不是要重写？」	→ 工具层是唯一耦合点。ToolDef 的契约是 parameters / allowedRoles / isWrite / confirmSummary / run —— 换掉 run 的实现就行，契约本身、权限声明、HITL 确认卡、前端全都不动。真正的工作量在契约之外的两处：① 字段语义对齐 —— 「可用库存」这个口径含不含在途、含不含质检中、扣不扣安全库存，三家 ERP 三个答案；② 延迟 —— 本地个位数毫秒变成真实 API 几百毫秒，会逼出并行调用、超时降级和部分失败处理。	→ 现在 15 个 run 全是纯函数、读同一个 DbSnapshot，替换点就是 registry.ts 里那 15 处；哪一行会变、哪一行不会变，可以现场指给你看。	→ 还有一件必须同时做的事：网页那条链路的三道闸现在全在浏览器端。飞书那条链路已经把身份判定搬到了服务端（resolveUser 查表，不由调用方传），算是先跑通了一半；真接 ERP 的那一天，整条信任边界都得搬过去。
9 四个角色算出来的数不一样？ 「同一个缺口，李娜看到 8 台，王强看到 4 8 台？」	→ 这不是 bug，这是权限过滤生效的证据。张伟 0 张风险订单 / 李娜 8 台缺口 · 1 张风险订单 / 王强与陈立 48 台缺口 · 2 张风险订单 —— 每个角色只在自己能看到的订单集合上跑 simulateDeliveryRisk，输入集合不同，输出当然不同。这一点我在演示里主动讲、不等对方发现：如果四个角色算出来都一样，那才是权限系统没生效。	→ 现场切三次角色、问同一个问题，三组数字当场出来。scopeOrders 决定输入集合，risk.ts 只在这个集合上做纯计算，两者职责不重叠。	→ 代价是跨角色对账时，人得知道自己看到的是「本人视角下的数」。所以每次回答都带 ScopeSummary 自报范围 —— 那句话不是装饰，是对账凭据。
10 出了问题怎么追溯？ 「这张采购单是谁批的、凭什么批的？」	→ auditOf() 在每次工具调用后落一条 auditLog：角色、用户、工具名、入参、结果、耗时，越权代办再额外打一个 override 标记。所以任何一次写入都能倒推出「哪个人、以什么角色、基于前面哪几次查询、在哪一秒点的批准」。它挂在执行路径上而不是 UI 上，绕不过去。	→ loop.ts:221 —— o.pushAudit(auditOf(name, args, r, ctx(), !!overrideNotice)), 位置在 executeTool 之后、emit 之前，任何一条成功执行都必然经过它。	→ 两个缺口我先说：① 被权限拒绝的调用当前没进 auditLog，而拒绝率恰恰要靠它算，这是上线前要补的一行；② 审计只读界面排在 V2 之后，现在只能在 devtools 里看。

不用等你问：这六条我自己先说

一个只讲优点的系统介绍，听起来像销售话术；主动划出边界，对方才知道哪些结论可以直接采信。飞书那条链路另有三条，见第 11 页。

1 网页链路的三道闸还在浏览器里

飞书那条链路的身份已经由服务端 resolveUser 查出来了；网页这条仍然是 toolsFor / executeTool / scope* 全在客户端，改 devtools 就能绕。

2 被拒绝的调用没进 auditLog

auditOf 只在成功执行后调用。而「人工确认拒绝率」这个核心指标，分子恰好就是这批没被记下来的调用。

3 服务端那一侧的审计直接丢弃

runAgentServerSide 里 pushAudit 是个空函数。只读前提下可以接受，但真接数据之前必须先落库。

4 第七类幻觉只能靠提示词

它不编数据，编「系统的行为」—— 「订单已归档至总部财务系统」。修法方向是让工具显式回 hiddenReason。

5 清单上的勾是近似值

loop.ts 的 stepIdx 每轮推进一格，不与 expectedTools 对齐。它表示「跑到第几轮」，不是「哪一步真做完了」。

6 会话记忆这个取舍我定错过

原本 2 轮 + 每轮截 600 字，追问链第三轮就断、长回答从中间被切。现在是 6 轮 —— 这是被真实使用打回来的。

横向读一行是一次完整应答；纵向读第四栏，是这个系统当前真实的能力边界。底部六条缺口按「发现它的代价」排序：越靠右，越是要等真实用户来打脸才会发现的那一类。

METRICS

14 评测体系：为什么先盯「工具选择准确率」

评测的目的不是给系统打一个分，而是让「这次改动到底是变好了还是变坏了」这个问题有一个不靠感觉的答案。
离线管回归、在线管价值；先说清楚它能抓什么，再说清楚它抓不到什么。



ROADMAP

15 V1 → V2：写触发条件，不写时间表

这一页的每一行都是一条「if 这件事发生 → then 必须做这个」，而不是一个季度计划。
第一列是我自己先说出来的缺口，最后一列是它现在还没做的真实原因。

V1 · 现在 可演示的产品原型 要证明的是：Agent 的决策链路能不能被约束住。	→	V2 · 下一版 能交给真实用户的最小版本 要证明的是：这套约束在真实数据和真实延迟下依然成立。		下面每一行的读法 不是「什么时候做」，而是「什么事发生了就必须做」。 路线图会过期，触发条件不会。一个没有触发条件的 V2 计划，本质上是许愿。
V1 现在是什么样	触发条件：什么时候必须动它	那时候做什么	先决条件 · 代价 · 为什么现在不做	
P0 · 阻塞上线 网页那条链路的三道闸仍然全在浏览器端，改 devtools 就能绕过去。飞书那条链路已经先走了一半：身份由服务端 resolveUser 查出来，不由调用方传。	→	任何一个真实用户、任何一份真实数据接进来的那一刻 —— 没有缓冲期。	→	把 toolsFor / executeTool / scope* 整体搬到服务端，浏览器只保留渲染；角色由服务端会话决定，前端不再持有 user 对象。飞书链路已经证明执行器搬得动：loop.ts 一行没改就在 Worker 里跑通了。
P0 · 新链路带来的 服务端执行器的审计直接丢弃：runAgentServerSide 里 pushAudit 是个空函数。	→	飞书那条链路开始承载只读之外的任何东西的那一刻 —— 或者有人问「谁在群里问过什么」。	→	把 auditLog 从内存数组换成一个可持久化的落点（KV / D1），两条链路共用同一个写入接口，服务端不再是特例。
P0 · 一行代码 auditOf() 只在成功执行之后调用；被权限拦下的、被人拒绝的调用，一条都没留痕。	→	想开始看「人工确认拒绝率」的那一天 —— 而它是四个在线指标里最灵敏的一个。	→	在 executeTool 的 PERMISSION_DENIED 分支、以及 HITL 的 userDeclined 分支各补一次 auditOf，用 outcome 字段区分成功 / 拒权 / 人拒。
V2 · 第一批 审计数据已经在落，但没有给人看的入口 —— 要开浏览器控制台，才答得出「这一单是谁批的」。	→	出现第一次需要事后复盘的写操作。	→	审计只读界面：按人 / 角色 / 工具 / 时间筛选，越权代办单独标色，每条能跳回当时那次会话的完整工具链。
V2 · 待触发 四个写工具走的是同一条 HITL：每一次写，人都要点一次。	→	写工具从 4 个涨到两位数，或出现「改日期」这类低风险高频写。	→	写操作分档：低风险默认执行、事后可撤销；高风险仍然必须确认。把确认卡留给真正不可逆的那一类。
V2 · 有前提 全局固定 qwen-plus / temperature 0.2，规划和执行用的是同一个模型。	→	离线 30 条评测集真正跑起来之后 —— 是之后，不是之前。	→	按任务分级路由：规划器和写操作留在强模型，单跳只读查询下沉到更快更便宜的模型。
V2 · 待触发 第七类幻觉 —— 它不编数据，编的是「系统的行为」—— 目前只能靠提示词顶住。	→	用户第一次因为一句「订单已归档至总部财务系统」，真的去找了 IT。	→	让工具在数据被裁剪时显式返回 hiddenReason，把「为什么查不到」从模型的想象变成工具的返回值。

V2 明确不做的三件事

「不做什么」和「做什么」是同一份路线图的两半 —— 只写要做的，那不是取舍，是许愿清单。

✘ 不做多 Agent 编排

现在的失败模式是「一个 Agent 查不准」，不是「一个 Agent 忙不过来」。多加一层编排，只会让「这一步错在哪」更难归因。

✘ 不急着加工具

15 个工具已经覆盖了演示里的全部问题。工具越多，工具选择准确率越难保。先把评测跑起来，再决定加哪一个。

✘ 不做全自动执行

把 HITL 去掉能让 demo 更顺，但这个产品的价值恰恰在那一次点击上 —— 不可逆的动作前面必须站一个人。

横向读一行：现状 → （什么事发生）→ 做什么 → （代价是什么）。三条 P0 是上线前的硬门槛，其余四条都要等一个明确的信号才动手。底部三条是主动排除项 —— 它们比要做的事更能说明这个产品想成为什么。

SUMMARY

16 决策总表：选了什么 · 代价是什么 · 什么时候我会改选

前十四页是「怎么做的」，这一页是「为什么这么做，以及在什么条件下我会推翻它」。
每行右边的页码，是这条决策在本册里的证据位置。

读法

每一行都是一次取舍：左边是我选了什么，中间是为这个选择付的钱，右边是什么信号出现时我会重新选。

选择

代价 · 放弃了什么

改选的触发信号

决策点	我选了什么	代价 · 放弃了什么	什么条件下我会改选
架构形态 P05	➡ Planner-Executor 两段式：先出一份用户能看见的计划，再逐步执行。	➡ 多一次 LLM 调用、多几秒延迟；计划一旦错，整条执行链跟着歪。	➡ 如果绝大多数问题都是单跳查询，两段式就是纯浪费 —— 退回单段 ReAct。
数据来源 P01	➡ 确定性种子 generateSeed(42) + 手工埋雷剧本，而不是接真实数据。	➡ 只覆盖种子里存在的情形；真实数据的脏值和缺字段，一条都测不到。	➡ 接真实 ERP 之后：种子库降级为回归测试夹具，不再充当演示数据源。
计算归属 P06	➡ 确定性计算写成纯函数（risk.ts 40 行），只把「读什么、怎么说」留给模型。	➡ 每一条新业务规则都要写代码，模型没法自己适应新场景。	➡ 出现真正无法用规则表达、且允许模糊的判断时（例如客户意向分级）。
工具粒度 P06	➡ 15 个窄工具（只读 11 / 写 4），一个工具对应一个业务问题。	➡ 工具一多，工具选择准确率就难保；每加一个都要补评测样本。	➡ 某一类问题稳定需要 3 次以上串联、且中间没有任何一步需要人介入时。
权限位置 P07	➡ 三道闸放在工具层（可见性 / 执行 / 数据范围），角色永远不进模型上下文。	➡ 网页那条链路的三道闸仍在浏览器端 —— 改 devtools 就能绕过去。	➡ 任何真实数据接进来的那一刻，整体搬到服务端。飞书链路已先搬了身份判定这一半。
幻觉治理 P08	➡ 六道代码防线加一道 UI 兜底，而不是靠提示词里多写几句「不要编造」。	➡ tool_choice:'required' 会让最简单的问题也强制走一次工具，多花一次调用。	➡ 模型端稳定支持可验证的引用之后，守卫 C 这类事后核对可以让位。
人在哪一步 P09	➡ 只拦写、不拦读；4 个写工具全部走人工确认，摘要由代码按实时库生成。	➡ 写的频次一高，确认就会退化成盲点击 —— 拦得越多，该拦的那次越拦不住。	➡ 写工具涨到两位数，或出现「改一个日期」这类低风险高频写时，改成分档。
失败呈现 P10	➡ 分类文案 + always 给一条出路，绝不把 String(e) 直接上屏。	➡ 分类枚举要人工维护；没被枚举到的新错误会掉进兜底文案里。	➡ 错误类型超出可枚举范围时，改成「错误码 + 可复制的诊断信息」。
模型选型 P12	➡ 全局固定 qwen-plus / temperature 0.2，规划与执行共用一个模型。	➡ 单跳查询也在付强模型的钱；输入 token 是输出的 10-30 倍。	➡ 离线评测集真正跑起来之后 —— 是之后 —— 再按任务分级路由。
会话记忆 P04	➡ 保留最近 6 轮问答，更早的轮次折叠成摘要。	➡ 上下文更长、更贵；摘要本身也可能丢掉关键指代。	➡ 这一条我已经改过一次：原本 2 轮 + 每轮截 600 字，追问链第三轮就断。
触达方式 P11	➡ 同一份执行器在 Worker 里再跑一遍，飞书群里 @ 一下就能问。	➡ 服务端那一侧没有审计、没有写权限，只读是它成立的前提。	➡ 这条链路要承载写操作的那一天 —— 那时候 HITL 必须在飞书里也成立，而不是靠回跳。

这一册里没有一个决策是免费的。

能说清代价、也能说清「什么时候我会改主意」的选择，才叫决策；说不清的，只是默认值。

所以这十一行里，最重要的是第三列和第四列 —— 第二列谁都写得出来，前两列合起来才证明这个选择是想过的。

十一条决策，横向读：决策点 → 我选了什么 → （代价）→ （改选信号）。第五、第七、第九行的改选条件已经在第 15 页有了明确的触发定义；最后一行是唯一一条已经被真实使用推翻过一次的决策。